

Ejercicios de Pthreads en C

De nivel básico a intermedio

PMD

24 de septiembre de 2026

Introducción

Los siguientes ejercicios están diseñados para practicar el uso de hilos POSIX (**pthread**) en C. Se cubren tres temas fundamentales:

- **Mutex** (`pthread_mutex_t`): exclusión mutua para proteger secciones críticas.
- **pthread_join**: sincronización para esperar la terminación de hilos.
- **Variables de condición** (`pthread_cond_t`): comunicación y espera entre hilos.

Se recomienda compilar con:

```
gcc archivo.c -o programa -lpthread
```

1. Nivel Básico

Ejercicio 1: Hola Mundo con Hilos y pthread_join

Escribe un programa que cree **5 hilos**. Cada hilo debe imprimir su identificador (recibido como argumento) y el mensaje "Hilo i ejecutándose". El hilo principal debe esperar a que **todos** los hilos terminen usando `pthread_join` antes de imprimir "Todos los hilos han terminado".

Objetivo: Practicar la creación de hilos y `pthread_join`.

Ejercicio 2: Suma de un arreglo con mutex

Dado un arreglo de enteros de tamaño $N = 1000000$ inicializado con valores aleatorios, crea **4 hilos**. Cada hilo suma una porción del arreglo y acumula el resultado en una variable global `suma_total`. Protege el acceso a `suma_total` con un `pthread_mutex_t`.

Objetivo: Comprender el uso básico de mutex para evitar condiciones de carrera.

Ejercicio 3: Contador compartido

Implementa un programa con **10 hilos**, donde cada hilo incrementa un contador global **100000** veces.

- Primero realiza el incremento **sin** mutex y observa el resultado final.
- Luego repite el experimento **con** mutex.

Explica la diferencia observada.

Objetivo: Visualizar experimentalmente el problema de las condiciones de carrera.

2. Nivel Intermedio

Ejercicio 4: Productor–Consumidor con buffer acotado

Implementa el clásico problema **productor–consumidor** usando:

- Un buffer circular de tamaño $B = 5$.
- **2 hilos productores** que insertan números enteros aleatorios.
- **2 hilos consumidores** que extraen y muestran los valores.
- Un `pthread_mutex_t` para proteger el buffer.
- **Dos variables de condición:** `no_lleno` y `no_vacio`.

Los productores se detienen después de insertar 50 elementos cada uno. El programa debe terminar cuando todos los elementos hayan sido consumidos.

Objetivo: Combinar mutex y variables de condición para sincronizar hilos.

Ejercicio 5: Barrera de sincronización con variables de condición

Implementa una **barrera** manual usando `pthread_mutex_t` y `pthread_cond_t`. La barrera debe tener las siguientes funciones:

```
1 typedef struct {
2     pthread_mutex_t mutex;
3     pthread_cond_t cond;
4     int count;
5     int total;
6     int generation;
7 } barrera_t;
8
9 void barrera_init(barrera_t *b, int total);
10 void barrera_wait(barrera_t *b);
11 void barrera_destroy(barrera_t *b);
```

Crea **6 hilos** que ejecuten 3 fases de trabajo. Cada hilo debe llamar a `barrera_wait()` al final de cada fase, de modo que ningún hilo avance a la siguiente fase hasta que todos hayan terminado la actual.

Objetivo: Entender el uso avanzado de variables de condición y generaciones.

Ejercicio 6: Cola de tareas (thread pool simplificado)

Implementa un **pool de 3 hilos trabajadores** que consuman tareas de una cola compartida. Cada tarea es una función simple (por ejemplo, imprimir un número y dormir 100 ms). El programa debe:

- a) Encapsular la cola con mutex y variables de condición.
- b) Permitir que el hilo principal agregue 20 tareas.
- c) Cuando ya no haya tareas y se indique el cierre (**shutdown**), los trabajadores deben terminar limpiamente.

Objetivo: Aplicar mutex y variables de condición a un patrón real de concurrencia.